

Agenda

- Boxing & UnBoxing
- Packages
- Access Modifiers
- Method Overloading
- this reference
- Types of Methods
- ~~Constructor Chaining~~
- ~~Array~~

Boxing & UnBoxing

- Converting value from primitive type to reference type is called as boxing
- Converting value from reference type to primitive type is called as unboxing

```
int num1 = 10;
Integer i1 = new Integer(num1); // boxing
Integer i2 = num1; // auto-boxing

Integer i3 = new Integer(20);
int num2 = i3.intValue(); // unboxing
int num3 = i3; // auto-unboxing
```

Packages

- Packages makes Java code modular. It does better organization of the code.
- Package is a container that is used to group logically related classes, interfaces, enums, and other packages.
- Package helps developer:
 - To group functionally related types together.
 - To avoid naming clashing/collision/conflict/ambiguity in source code.
 - To control the access to types.
 - To make easier to lookup classes in Java source code/docs.
- Java has many built-in packages.
 - java.lang -> Integer, String, System
 - java.util -> Scanner
 - java.io -> PrintStream, Console
 - java.sql -> Connection, Statement
- To define a type inside package, it is mandatory write package declaration statement inside .java file.

- Package declaration statement must be first statement in .java file.
- Types inside package called as packaged types; while others (in default package) are unpackaged types.
- Any type can be member of single package only.
- It is standard practice to have multi-level packages (instead of single level). Typically package name is module name, dept/project name, website name in reverse order.
- When compiled, packages are created in form of directories (and sub-directories).

```
package com.sunbeaminfo.kdac
```

Access Modifiers

- For class
 - 1. default
 - 2. public
- For class members
 - 1. private
 - only within the class directly
 - 2. default (package level private)
 - in same class directly
 - in all the classes in the same package on class object
 - 3. protected
 - in same class directly
 - in all the classes in the same package on class object
 - in subclasses directly
 - 4. public
 - are visible every where.

Access Modifier	In same Package			In Different Package	
	Same class	Other Class	Sub Class	Other class	Sub Class
private	A	NA	NA	NA	NA
default	A	A	A	NA	NA
protected	A	A	A	NA	A
public	A	A	A	A	A

Difference between protected and default

- Default access restricts visibility to only classes within the same package. This allows you to encapsulate implementation details that are not intended to be accessed by classes outside the package.
- Protected access, on the other hand, allows access by subclasses (regardless of package) and by other classes within the same package.

- If you want to hide implementation details from all classes, including subclasses, default access provides stricter encapsulation.

Method Overloading

- Defining methods with same name but different arguments(signature) is called as method overloading
- Arguments can differ in one of the following ways
 1. No of parameters should be different
 2. If no of parameters are same then their type of parameters should be different
 3. If no and type are same then the order of parameters should be different
- Count (no of parameters)

```
static int multiply(int a, int b) {  
    return a * b;  
}  
static int multiply(int a, int b, int c) {  
    return a * b * c;  
}
```

- type of parameter

```
static int square(int x) {  
    return x * x;  
}  
static double square(double x) {  
    return x * x;  
}
```

- Order of parameters

```
static double divide(int a, double b) {  
    return a / b;  
}  
static double divide(double a, int b) {  
    return a / b;  
}
```

- Note that return type is NOT considered in method overloading.

this Reference

- "this" is implicit reference variable that is available in every non-static method of class which is used to store reference of current/calling instance

- Whenever any non-static method is called on any object, that object is internally passed to the method and internally collected in implicit "this"
- "this" is constant within method i.e. it cannot be assigned to another object or null within the method.
- Using "this" inside method (to access members) is optional.
- However, it is good practice for readability.
- In a few cases using "this" is necessary like.

1. Unhide non-static fields from local fields i.e to differentiate between local variable and field of the class.

```
double height; // "height" field
void setHeight(double height) { // "height" parameter
    this.height = height;
}
```

2. Constructor chaining
3. Access instance of outer object.

Assignment

- Create the date class with a ctor that initializes the state with values 1,1,1900. provide the accept and display methods using this reference. Create 2 objects of date class and test them.
- Also try creating a menu driven code(optionl)